

Struct(ures)

FUNDAMENTALS OF PROGRAMMING

Bùi Duy Đăng

bddang@fit.hcmus.edu.vn

Plan for today

- Motivation
- Concept
- How to use
- Member functions
- Exercises

Motivation

- An information of **1** student

StudentID

// What types of data?

Name

// What types of data?

Birthday

// What types of data?

Gender

// What types of data?

Score_Intro2Program

// What types of data?

Score_Intro2InforTech

// What types of data?

Motivation

- An information of **1** student

StudentID	// char[9] e.g., 23127999
Name	// char[100]
Birthday	// char[11] e.g., 1900/01/01
Gender	// char
Score_Intro2Program	// float/double
Score_Intro2InforTech	// float/double

Motivation

- An information of **1** student

StudentID	// char[9] e.g., 23127999
Name	// char[100]
Birthday	// char[11] e.g., 1900/01/01
Gender	// char
Score_Intro2Program	// float/double
Score_Intro2InforTech	// float/double

- How to store the student information?

Motivation

- An information of **1** student

StudentID	// char[9] e.g., 23127999
Name	// char[100]
Birthday	// char[11] e.g., 1900/01/01
Gender	// char
Score_Intro2Program	// float/double
Score_Intro2InforTech	// float/double

- How to store the student information? → Using many variables (feasible?)

Motivation

- An information of **1** student

StudentID // char[9] e.g., 23127999

Name // char[100]

Birthday // char[11] e.g., 1900/01/01

Gender // char

Score_Intro2Program // float/double

Score_Intro2InforTech // float/double

- How to store the student information? → Using many variables
- How to store **n** student information?

Motivation

- An information of **1** student

StudentID	// char[9] e.g., 23127999
Name	// char[100]
Birthday	// char[11] e.g., 1900/01/01
Gender	// char
Score_Intro2Program	// float/double
Score_Intro2InforTech	// float/double

- How to store the student information? → Using many variables
- How to store **n** student information? → Using **struct**

Concept

- Structure is a **collection of variables** of **different data types** under **a single name**.

```
struct StudentInfo{  
    char ID[9];  
    char name[100];  
    char BirthDay[11];  
    char gender;  
    float score_Intro2Program;  
    float score_Intro2InforTech;  
};
```

Syntax

```
struct name{  
    DataType1 MemberName1 ;  
    DataType2 MemberName2 ;  
    DataType3 MemberName3 ;  
    . . .  
    DataTypeN MemberNameN ;  
} ;
```

Example

```
struct StudentInfo{  
    char ID[9];  
    char name[100];  
    char BirthDay[11];  
    char gender;  
    float score_Intro2Program;  
    float score_Intro2InforTech;  
};
```

Example

```
struct Point2D {  
    float x;  
    float y;  
};
```

```
struct Point3D {  
    float x;  
    float y;  
    float z;  
};
```

Example

```
struct Line2D{  
    Point2D start;  
    Point2D last;  
};
```

```
struct Line3D{  
    Point3D start;  
    Point3D last;  
};
```

How to use

Declaration

- Structures can be used as **user-defined data types**.

→ The uses are similar to other data types.

```
StudentInfo student1, student2;
```

```
Point2D point1, point2;
```

```
Line3D line1, line2;
```

```
// using arrays
```

```
StudentInfo sv[100]; // Arrays of 100 StudentInfo
```

```
Point2D points[100];
```

```
Line3D lines[100];
```

Initialization

- The way is similar to variable and array declaration

// note the order of members in struct

```
StudentInfo student1 = {"24127999", "Nguyen Van  
A", "01/01/2006", 'T', 9.5, 9};
```

```
Point2D point2D_point1 = {-23, 11};
```

```
Line3D line3D_line1 = {{0,0,0}, {1,1,1}};
```


Member Access

- Members in `struct` can be accessed via **dot operator** (`.`)

```
Point2D point1 = {-23, 11};
```

```
Point2D point2;
```

```
point2.x = 1.1;
```

```
point2.y = 2.2;
```

```
cout << "Point1(" << point1.x << " ; " <<  
point1.y << ")\n";
```

```
cout << "Point2(" << point2.x << " ; " <<  
point2.y << ")\n";
```

Structure Size

- Using the operator `sizeof` to get a size of a struct

```
cout << "Size of struct StudentInfo: " <<  
sizeof(StudentInfo) << endl;
```

```
cout << "Size of struct Point2D : " <<  
sizeof(Point2D) << endl;
```

```
cout << "Size of struct Line3D: " << sizeof(Line3D)  
<< endl;
```

Assignment Operator (=)

- The values of all members in one structure can be **automatically** assigned to another structure using the **assignment operator (=)** when the **two structures are of the same types**

→ Do not need to manually assign each members

E.g.,

```
Point2D point1;
```

```
Point2D point2;
```

```
point1.x = -32; point1.y = 11; // using dot operator
```

```
point2 = point1; // automatically assign
```

Passing Structures to Functions

- The way is similar to “passing datatypes to functions”

E.g.,

```
void Print(Point2D p) ;
```

```
void Input(StudentInfo &person) ;
```

```
void InputStudentArray(StudentInfo students[100]) ;
```

```
float calDistance(Point2D p1, Point2D p2) ;
```

```
float calDistance(const Line2D& line) ;
```

Passing Structures to Functions

```
void Print(Point2D p) {  
    cout << "(" << p.x << " ; " << p.y << ")";  
    cout << endl;  
}  
  
void Input(StudentInfo &person) {  
    cin.getline(person.ID, 9);  
    cin.getline(person.name, 9);  
    // ...  
    cin >> person.score_Intro2InforTech;  
}
```

Passing Structures to Functions

```
float calDistance(Point2D p1, Point2D p2) {  
    t1 = (p2.x - p1.x) * (p2.x - p1.x);  
    t2 = (p2.y - p1.y) * (p2.y - p1.y);  
    return sqrt(t1 + t2);  
}
```

```
float calDistance(const Line2D& line) {  
    t1 = pow((line.last.x - line.start.x), 2);  
    t2 = pow((line.last.y - line.start.y), 2);  
    return sqrt(t1 + t2);  
}
```

Member functions

Member functions

- Functions can be put **inside** a struct and **used as members** of the struct
- Purposes:
 - **Initialization** (constructor)
 - **Methods**

Initialization (Constructor)

- Name of functions with the same structure name

```
struct Point2D{  
    float x;  
    float y;  
    Point2D() { // constructor  
        x = 0;  
        y = 0;  
    }  
};
```

Methods

- The operations (methods) that a structure (variable) can do.

```
struct Point2D{  
    float x;  
    float y;  
    Point2D();  
    void Input();           // Get points from users  
    void Output();         // Print out the coordinates  
};
```

Methods

```
struct Point2D{  
    ...  
    void Input() {  
        cout << "input X: ";  
        cin >> X;  
        cout << "input Y: ";  
        cin >> Y;  
    }  
};
```

Methods

```
struct Point2D{  
    ...  
    void Output() {  
        cout << "(" << X << " "; << Y << ")";  
        cout << endl;  
    }  
};
```

Methods

Another way to define member functions is to use operator **::**

```
void Point2D::Input () {  
    cout << "input X: ";  
    cin >> X;  
    cout << "input Y: ";  
    cin >> Y;  
}  
  
void Point2D::Output () {  
    cout << "(" << X << " ; " << Y << ") \n";  
}
```

Function Member Access

- Function members in `struct` can be accessed via **dot operator** (`.`)

```
const int n = 10;
Point2D Point2DArray[n];
for (int i = 0; i < n; i++) {
    cout << "*** Point " << i + 1 << endl;
    Point2DArray[i].Input();
}
for (i = 0; i < n; i++)
    Point2DArray[i].Output();
```

Other topics

- Size of (`sizeof`) a struct depends on the order of the datatypes of members in the struct
- Union
- Constructor with parameters
- Copy constructor

Exercises

Fractions

- Done • Define a `struct` with `Fraction` data type (Fraction)
- Done • Input/Output a fraction
- Done • Reduce a fraction
- Done • Calculate addition, subtraction, multiplication, division of two fractions
- Done • Check if a fraction is simplified
- Done • Check whether a fraction is positive or negative
- Done • Compare two fractions

Exercises

Points in Oxy (2D)

- Define a `struct` with `Point` 2D data type (`Point`)
- Input/Output a point
- Calculate a distance of two points
- Find symmetric point from the X-axis/ the origin (0, 0)

Rectangles

- Define a `struct` with `Rectangle` data type (`Rectangle`)
- Input/Output a rectangle
- Calculate the perimeter, area of a rectangle

Exercises

Dates

- Done • Define a `struct` with `Date` data type (Date)
- Done • Input/Output a point (day, month, year)
- Done • Check if it is a leap year
- Done • Calculate the **ordinal number of a day** based on the **input date**
- Done • Find a date that is **k** days before/after the **input date**
- Done • Calculate the number of days between two dates
- Done • Compare two dates

Exercises

Fraction Arrays

- Done Input/Output n fractions
- Done Reduce all fractions
- Done Count the number of positive/negative fractions in the array
- Done Find the first positive fraction in the array
- Done Find the largest/smallest fraction in the array
- Done Sort the array in an ascending/descending way

Exercises

(2D) Point Arrays

- Done • Input/Output n points
- Done • Count the number of points where the X-axis value is positive for each point.
- Done • Find the points where the X-axis value is largest/smallest
- Done • Find the points whose distance to the origin (0, 0) is smallest